Student Record & Academic Management Platform

Automated CI/CD Pipeline with Jenkins, Docker and Kubernetes

Technical Project Report

Submitted By:

Name: Shubham Vijay Deshmukh

Department: AI and Data Science

PRN:2125UDSM1027

GithubLink:

https://github.com/shubhamdeshmukh1027/student-academic-platform

1. Introduction and Objective

This project implements an automated DevOps delivery pipeline for the Enterprise Student Record and Academic Management Platform. A Flask REST API is developed, tested, linted, built into a Docker image by a Jenkins pipeline, and deployed on a Kubernetes (Minikube) cluster with three replicas. The work follows the Library Platform case study from the solution document, adapted to the student domain.

1.1 Project Summary

Project	Student Record & Academic Management Platform
Entities	Students, Faculty, Courses, Enrollments, Grades
Business rule	Maximum 18 credits per semester per student
Application stack	Python, Flask, Gunicorn, pytest 8.2.1, pytest-cov, flake8
Automation tools	Git/GitHub, Jenkins 2.580.1 (declarative pipeline), Docker, Kubernetes with Minikube v1.39.0
Environment	Ubuntu 24.04 on Windows WSL2 (local Python 3.12; Docker image on Python 3.11)
Container image	student-academic-platform:v1.0.0
Kubernetes service	NodePort 30090, 3 replicas

1.2 Deliverables Checklist

#	Deliverable	Evidence in this report	Figures
1	GitHub repository	app/, database/, tests/, k8s/, Dockerfile, Jenkinsfile	1, 2

2	Branching and PR evidence	Git graph, branches, 6 merged pull requests, commit history	4 to 7
3	Jenkins pipeline and JUnit report	Stage view, console log (SUCCESS), 14 tests passing	10 to 12
4	Docker artifacts	Image build, docker images, docker ps with running container and health check	13 to 15
5	Kubernetes verification and service access	3 healthy replicas, self-healing after pod deletion, service URL and health check	18 to 22
6	Technical report	This document	-

2. Repository Structure

The repository follows the standard layout from the solution document: application code, database layer, tests, Kubernetes manifests, Dockerfile and Jenkinsfile at the root.

```
student-academic-platform/
  app/              routes, services, models
  database/         in-memory database with seed data
  tests/            pytest unit and API tests
  k8s/              deployment.yaml, service.yaml
  Dockerfile, .dockerignore, Jenkinsfile
  requirements.txt, run.py, .gitignore, README.md
```



Figure 1: Project files in the working directory

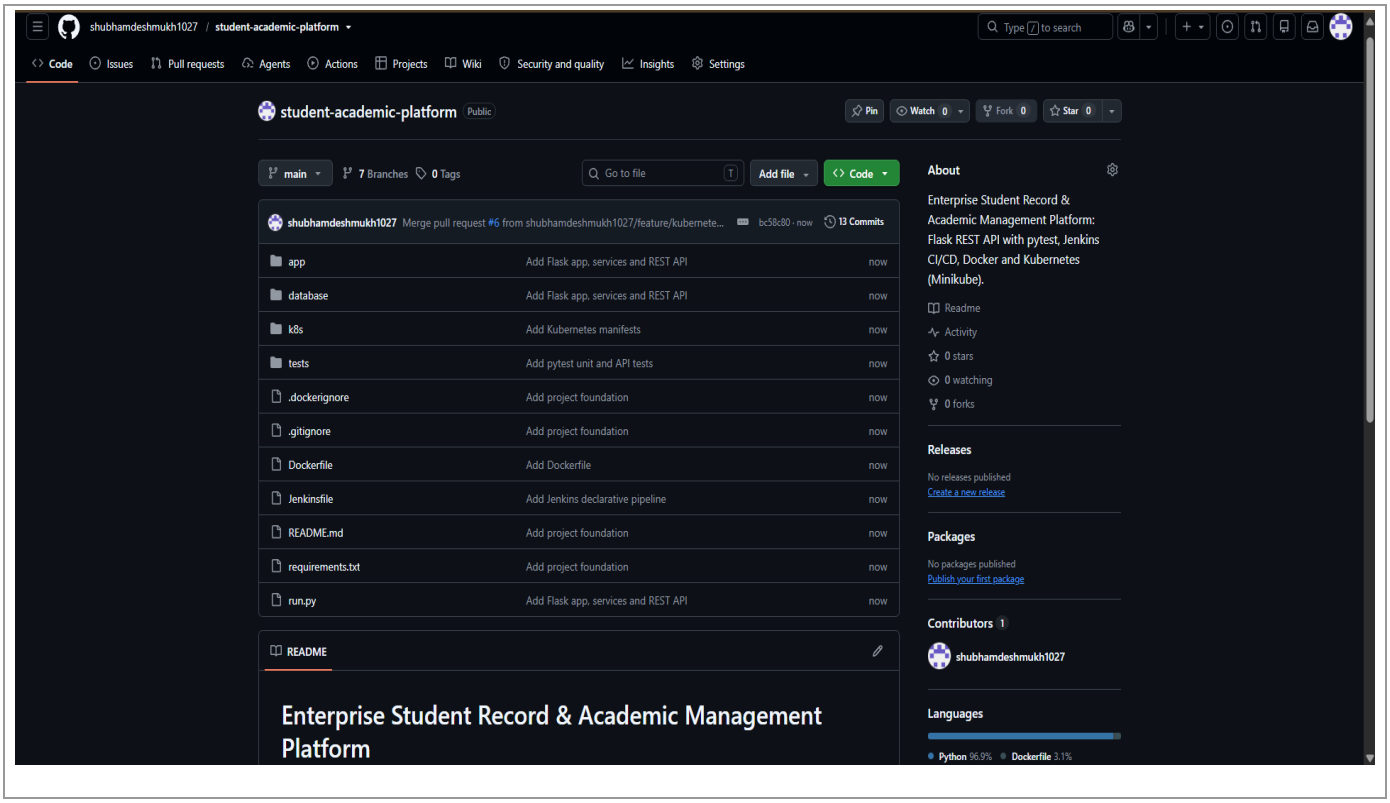


Figure 2: GitHub repository (public) with app, database, k8s, tests, Dockerfile and Jenkinsfile

3. Environment Setup

The environment is Ubuntu 24.04 running on WSL2. Python 3, pip, venv, Git and curl were installed with apt-get, and a Python virtual environment was used for the project dependencies (Flask, pytest, pytest-cov, gunicorn, flake8).

```
sudo apt-get update && sudo apt-get install -y python3 python3-pip
python3-venv git curl
python3 -m venv venv && source venv/bin/activate
```

```
pip install -r requirements.txt
```

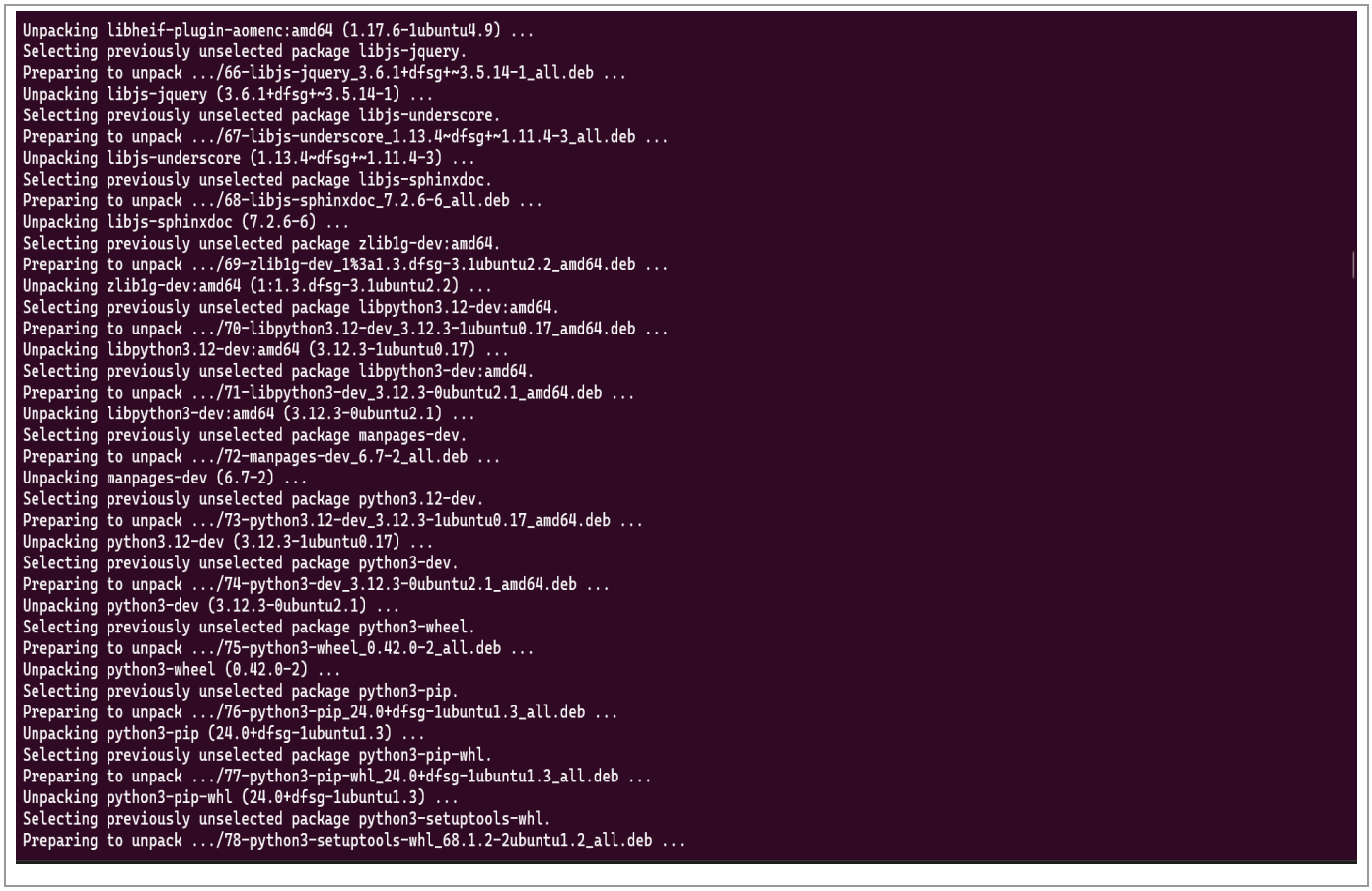


Figure 3: Package installation on Ubuntu (WSL2)

4. Git Workflow and Pull Requests

Every topic was developed on its own branch and merged into main through a pull request, so the history shows the progression from foundation to Kubernetes orchestration.

PR	Branch	Scope
#1	feature/foundation	Project foundation, .gitignore
#2	feature/core-app	Flask app, services and REST API
#3	feature/automated-tests	pytest unit and API tests
#4	feature/jenkins-ci	Jenkins declarative pipeline
#5	feature/dockerization	Dockerfile
#6	feature/kubernetes-orchestration	Kubernetes manifests

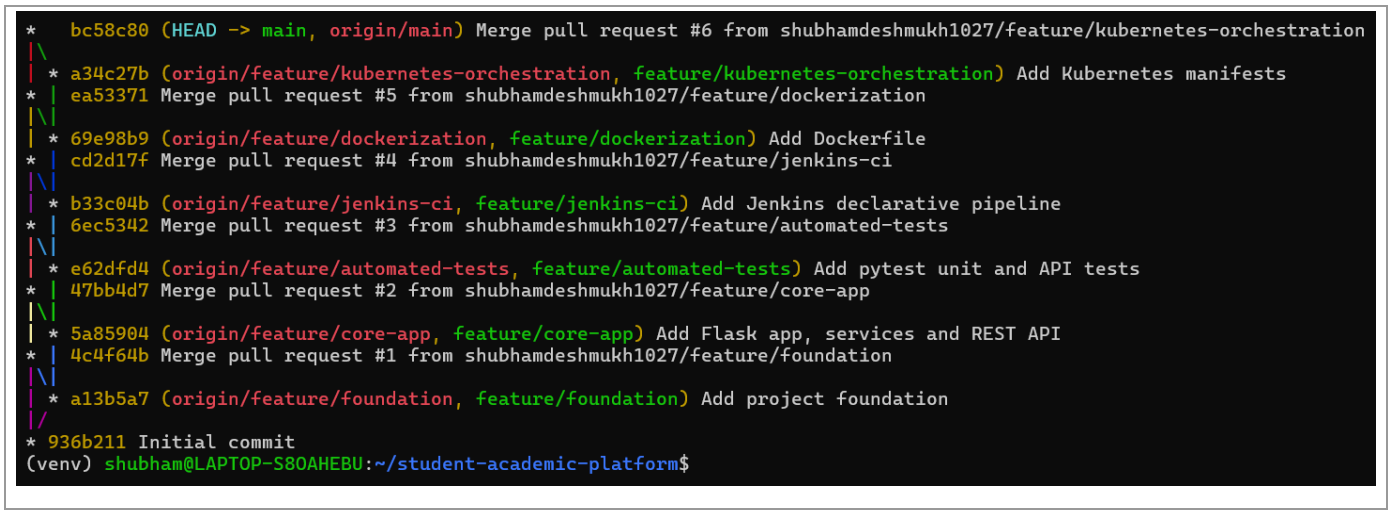


Figure 4: Git commit graph showing branch progression and merges into main

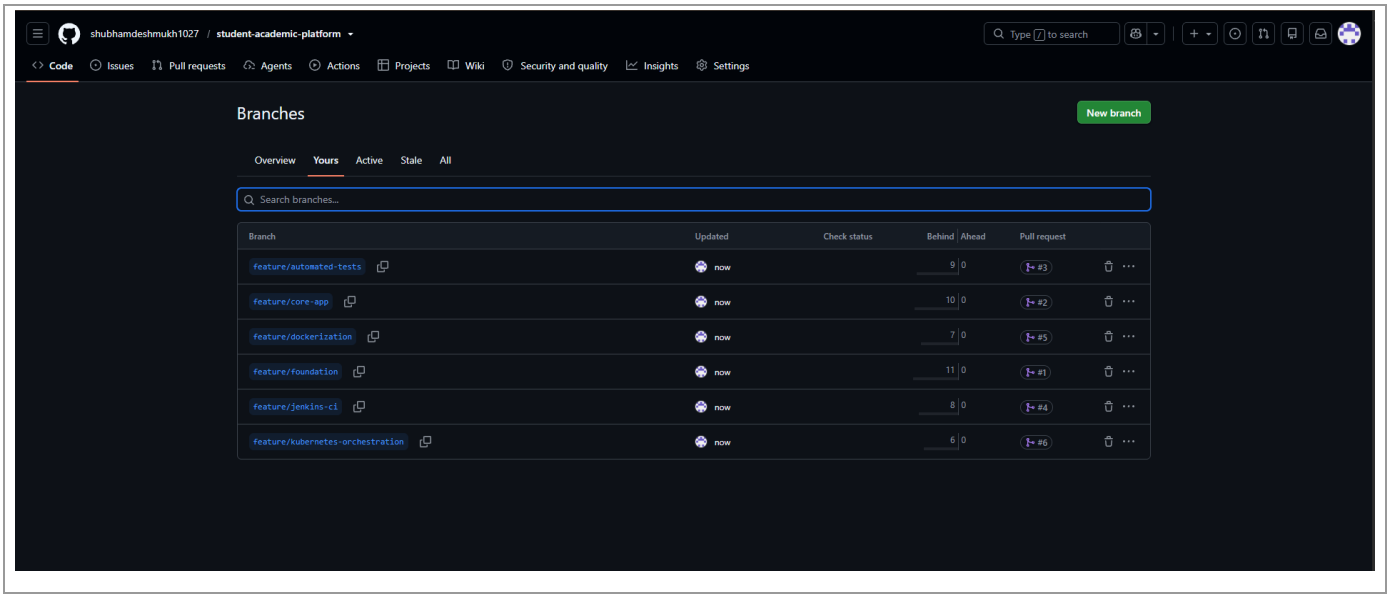


Figure 5: Feature branches with their linked pull requests

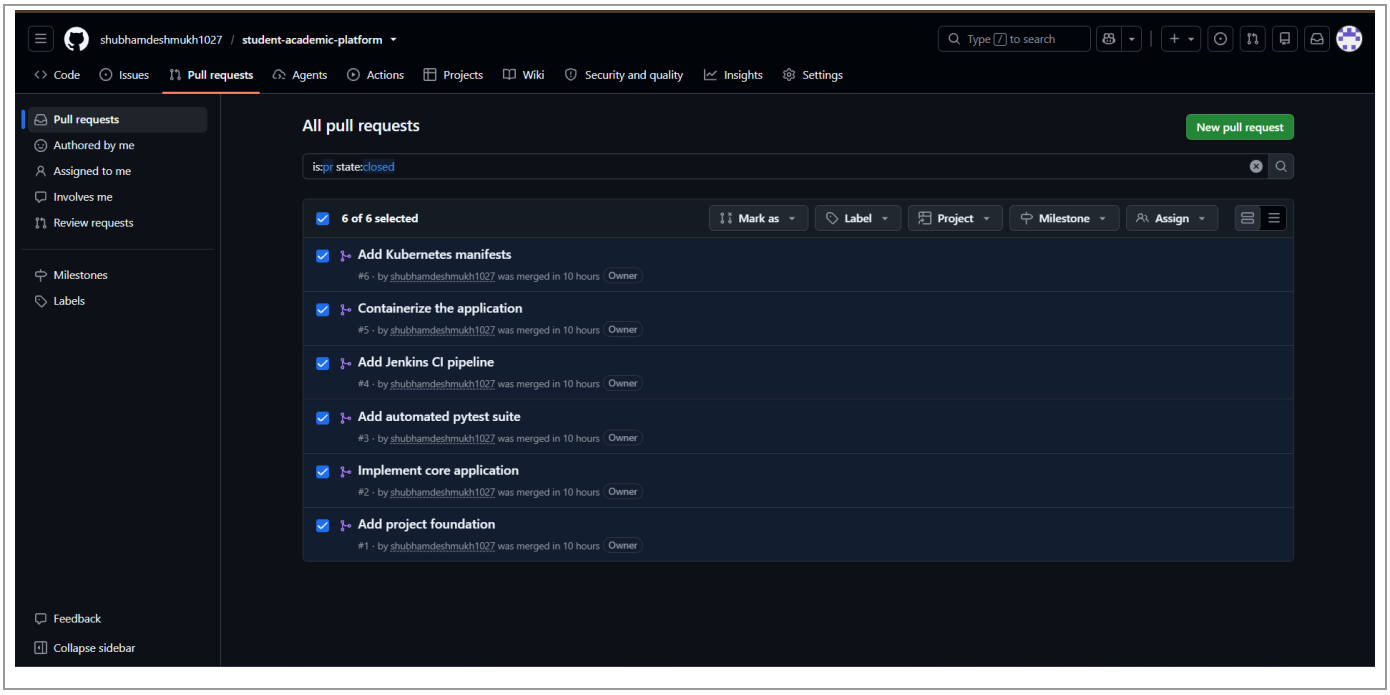


Figure 6: Six pull requests merged (closed) on GitHub

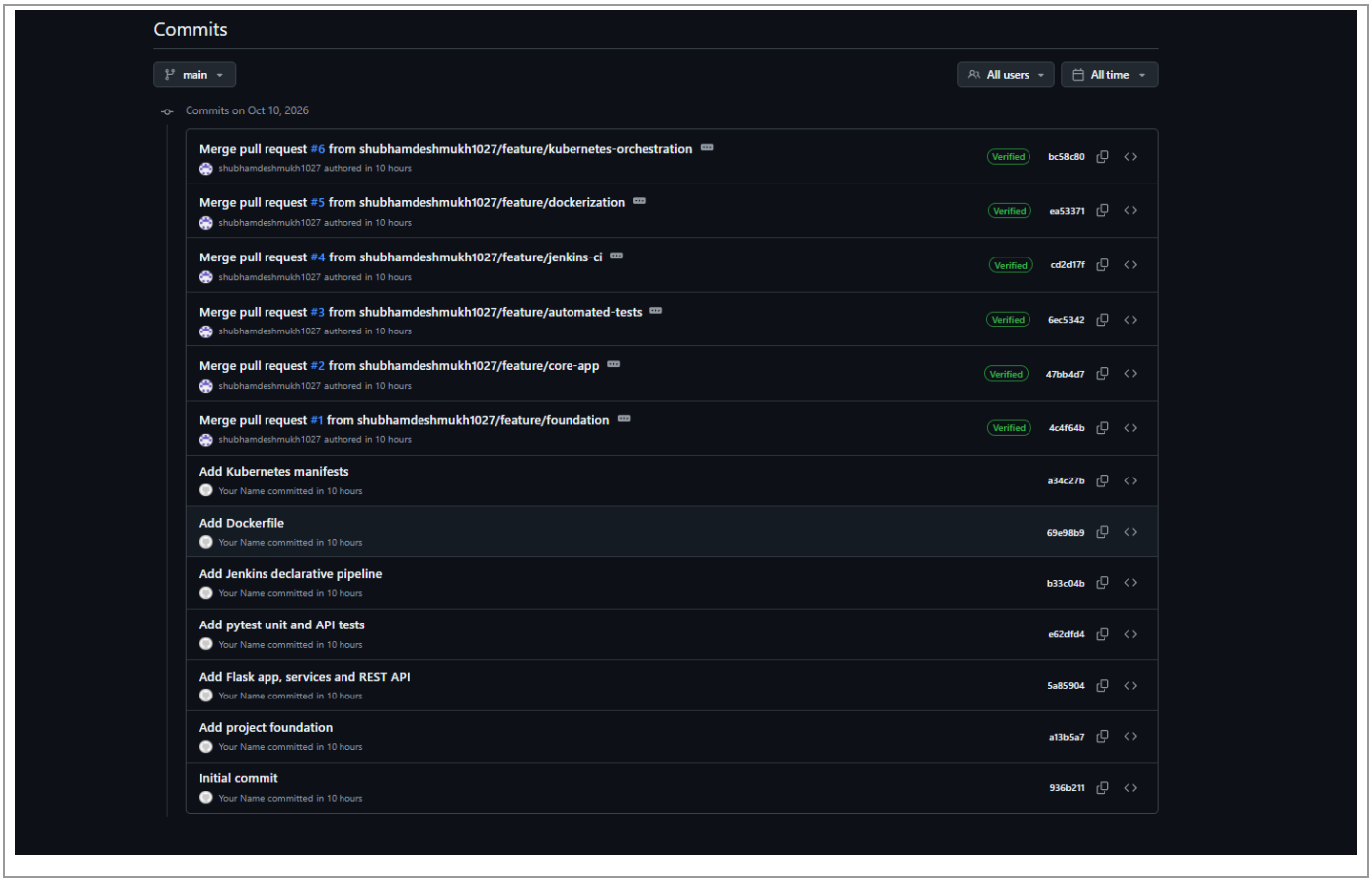


Figure 7: Commit history on main

5. Application, Automated Tests and Linting

The Flask application exposes a health endpoint (/api/health) plus endpoints for students, courses, enrollment and grades. The business logic enforces unique enrollments, schedule conflict checks and the 18-credit semester limit, and calculates GPA from grades.

5.1 Test Suite

The suite has 14 tests across three files: test_api_endpoints.py (health, list and create students, enroll and GPA, unknown student returns 404), test_enrollment_gpa.py (credits update, duplicate enrollment blocked, schedule conflict blocked, credit limit enforced, GPA calculation, invalid grade) and

test_student_service.py (add student, duplicate rejected, missing fields rejected). All 14 tests passed with 87% code coverage.

```
pytest -v --cov=app --cov-report=term-missing --
junitxml=reports/test-results.xml tests/
```

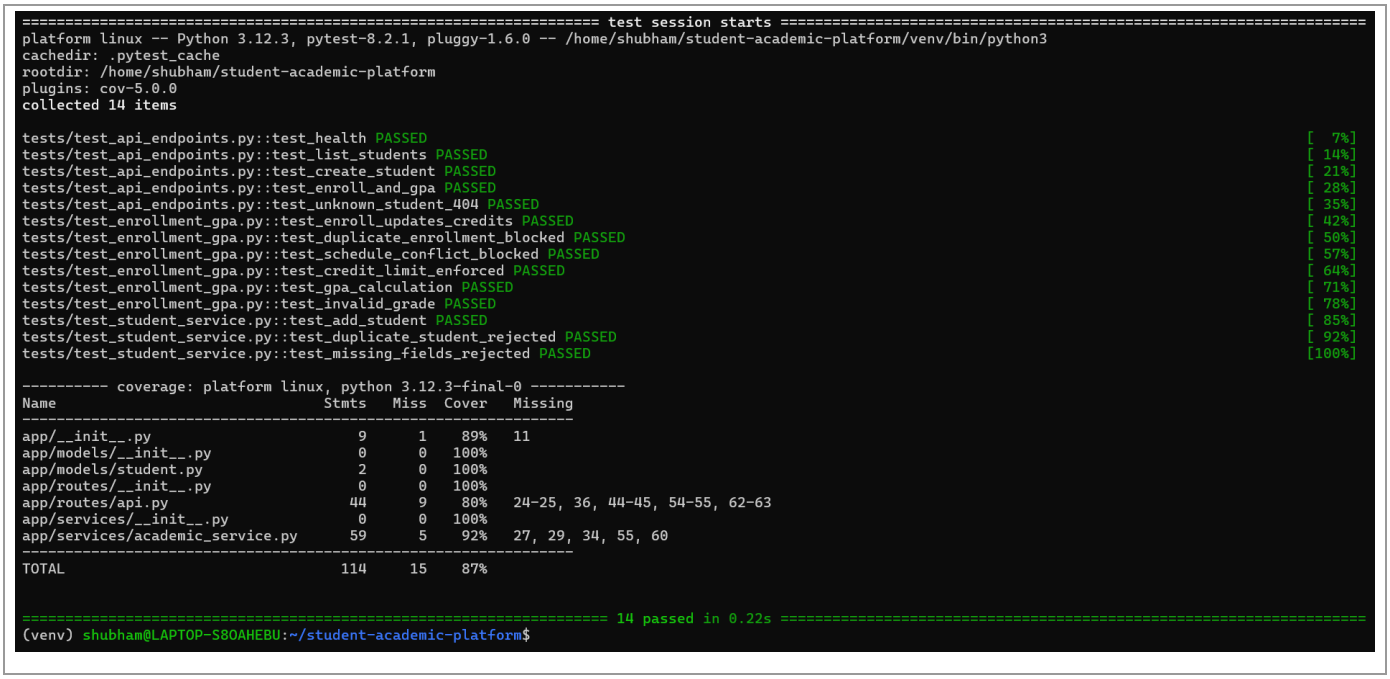


Figure 8: pytest run: 14 passed, 87% coverage

5.2 Code Quality

flake8 reported no issues, which is also enforced as a gate in the Jenkins pipeline.

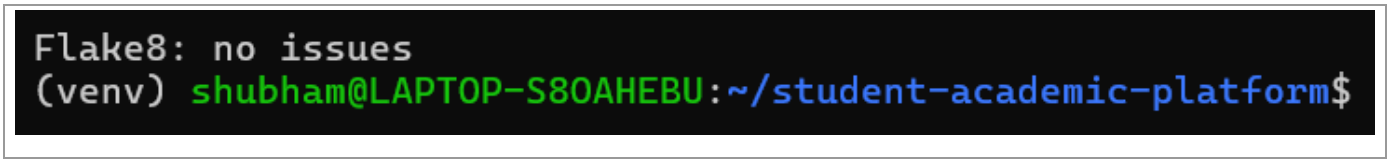


Figure 9: flake8: no issues

6. Jenkins CI Pipeline

Jenkins 2.580.1 was installed on Ubuntu with Java 17, unlocked with the initial admin password, and configured with the suggested plugins. A Pipeline job named student-platform-pipeline reads the Jenkinsfile from the GitHub repository (Pipeline script from SCM). Build #1 completed successfully in 54 seconds.

Stage	Purpose
Checkout SCM / Checkout	Fetch the source code from GitHub
Install Dependencies	Create the virtual environment and install requirements.txt (10 s)
Lint	Run flake8 as a code quality gate
Test	Run pytest with coverage and produce the JUnit XML report
Docker Build	Build the image student-academic-platform:v1.0.0 (30 s)
Post Actions	Publish JUnit results and print "Pipeline succeeded"

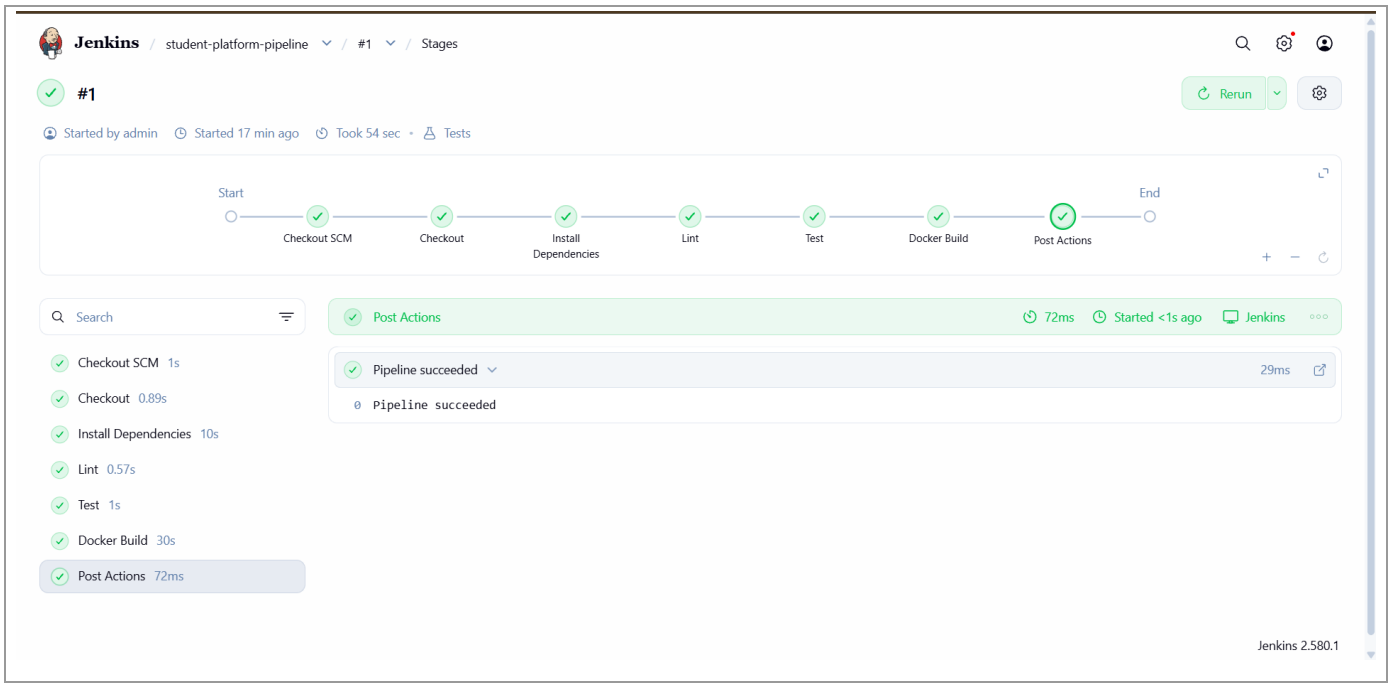


Figure 10: Jenkins pipeline stage view: all stages passed (Build #1)

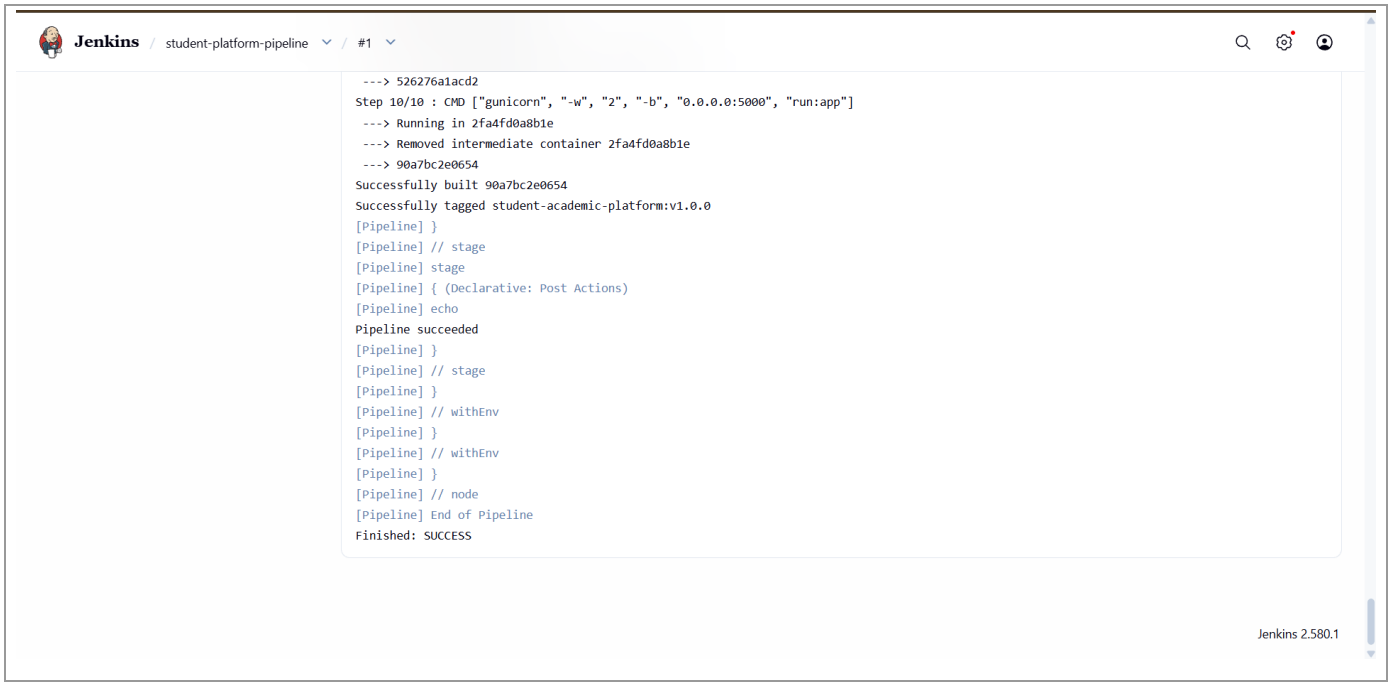


Figure 11: Jenkins console output ending with Finished: SUCCESS

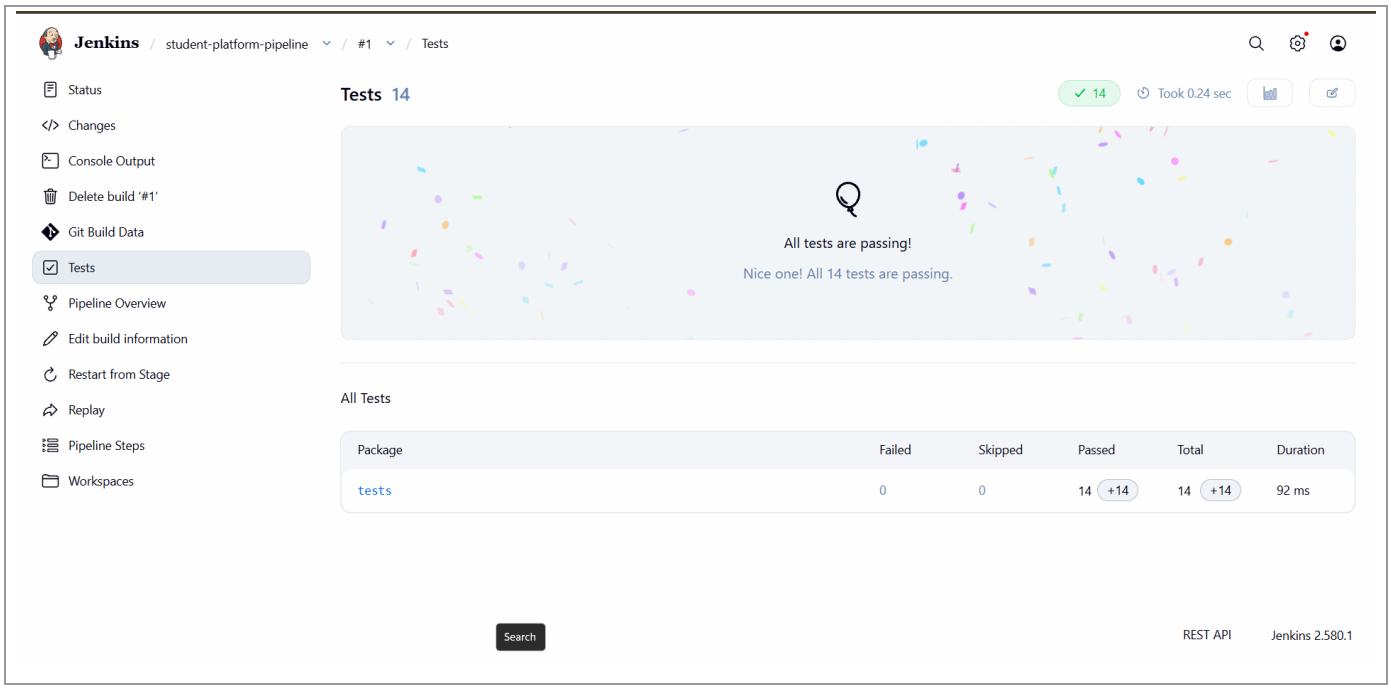


Figure 12: Jenkins JUnit test report: all 14 tests passing

7. Docker Containerization

The Dockerfile uses python:3.11-slim, sets PYTHONDONTWRITEBYTECODE and PYTHONUNBUFFERED, installs the requirements, copies the project, creates and switches to a non-root user (appuser), exposes port 5000 and starts the app with Gunicorn (gunicorn -w 2 -b 0.0.0.0:5000 run:app).

```
docker build -t student-academic-platform:v1.0.0 .
docker images
docker run -d -p 5000:5000 --name student-container student-
academic-platform:v1.0.0
docker ps
curl http://localhost:5000/api/health
```

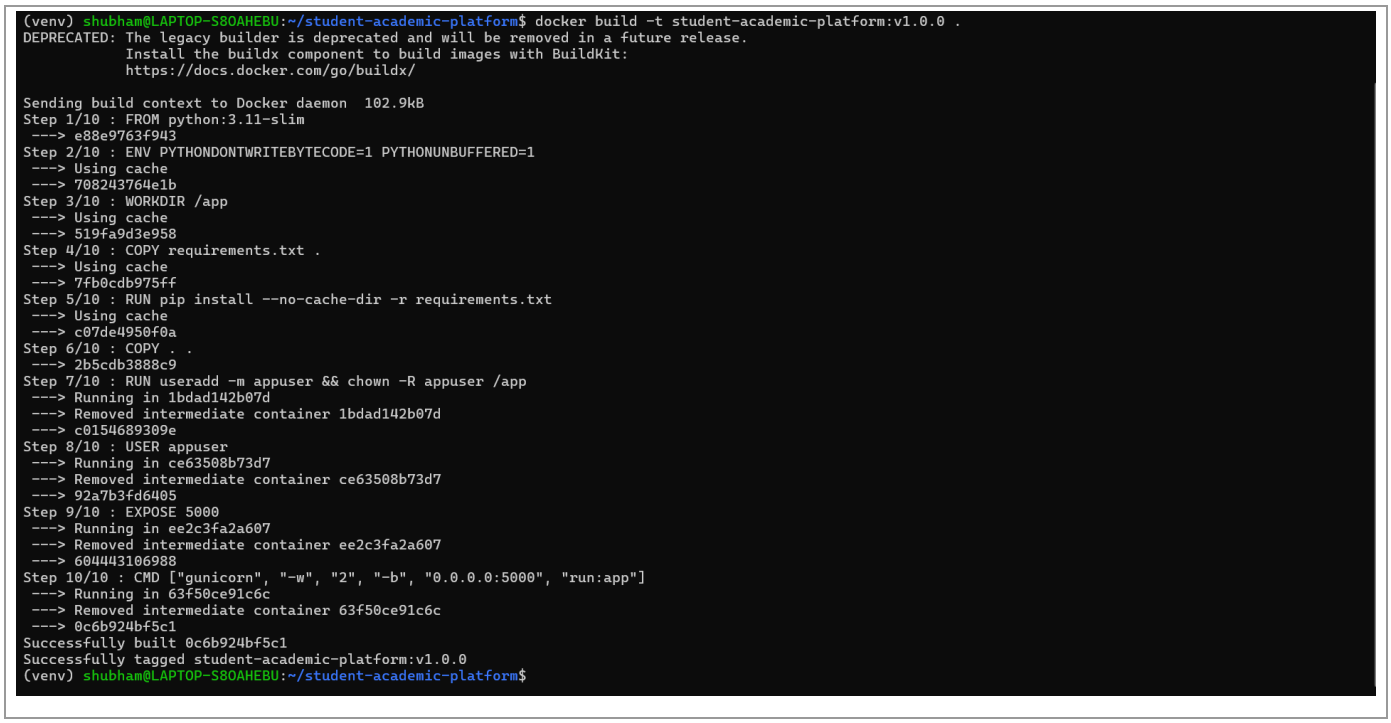


Figure 13: Docker image build completed and tagged v1.0.0

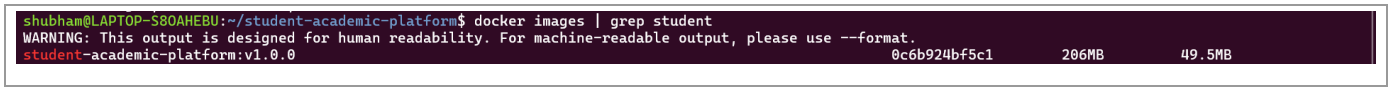


Figure 14: docker images: student-academic-platform:v1.0.0 (206 MB)

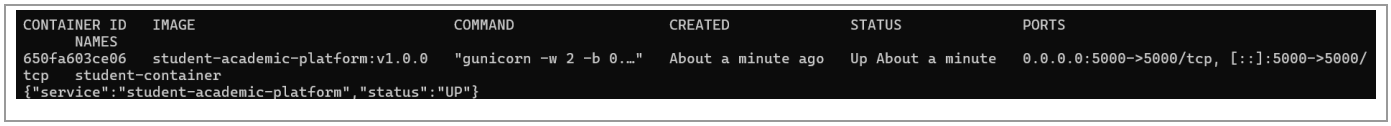


Figure 15: Running container (docker ps) and health check response

The container showed status Up with port 5000 mapped, and /api/health returned {"service":"student-academic-platform","status":"UP"}.

8. Kubernetes Orchestration on Minikube

Minikube v1.39.0 and kubectl v1.37.1 were installed, and a single-node cluster was started with the Docker driver (Kubernetes v1.37.0). Because the deployment uses imagePullPolicy: Never, the image was built inside Minikube's Docker daemon.

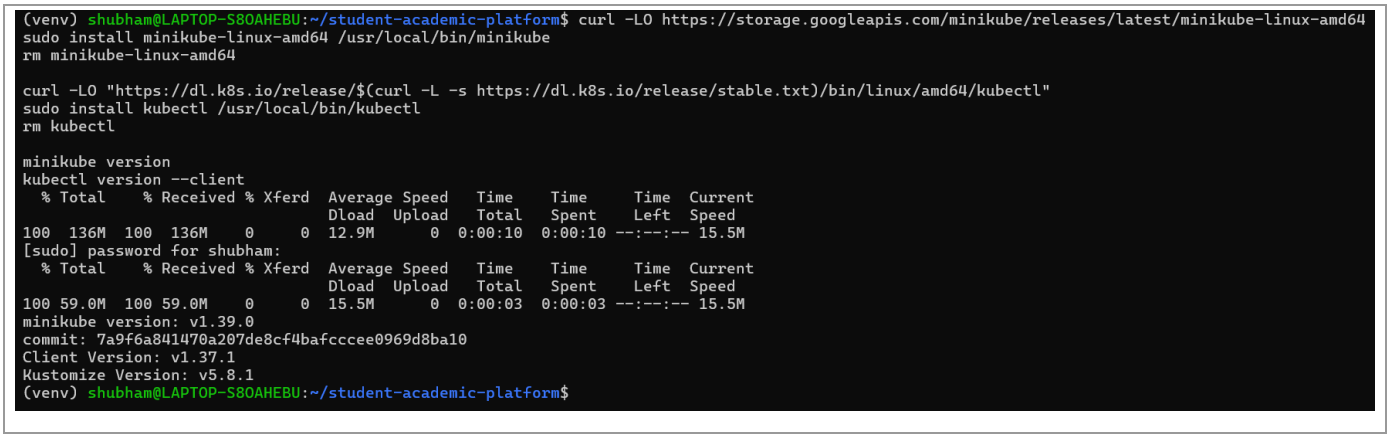


Figure 16: Minikube and kubectl installation with version output

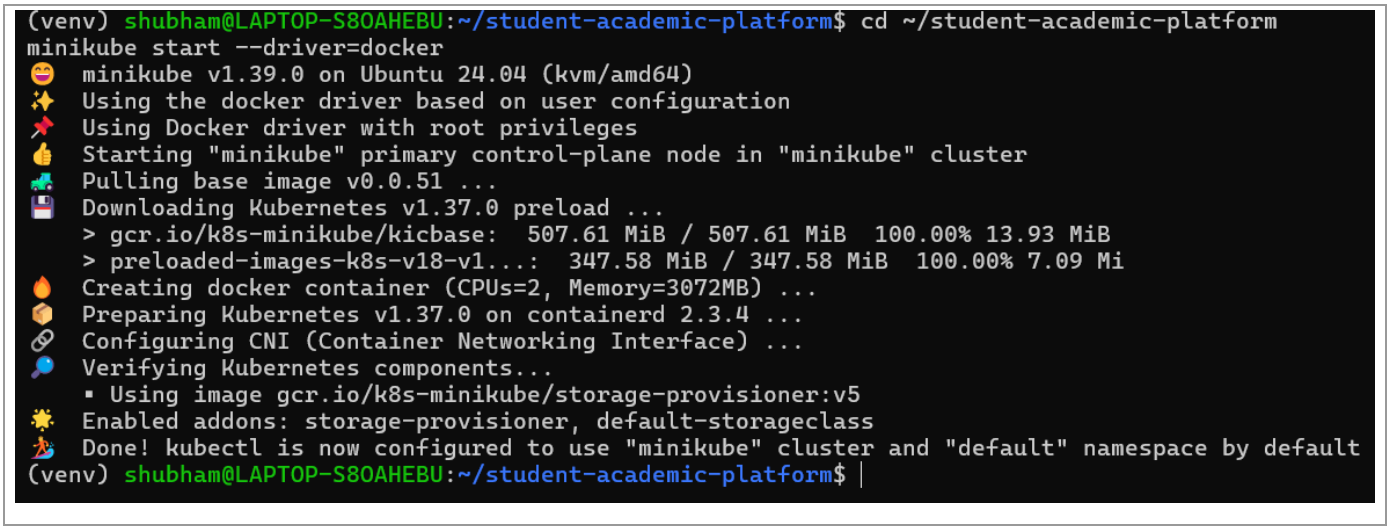


Figure 17: Minikube cluster started with the Docker driver

```
eval $(minikube docker-env)
docker build -t student-academic-platform:v1.0.0 .
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

8.1 Deployment Verification

Three pods of student-platform-deployment are 1/1 Running with zero restarts, and the service student-service is exposed as NodePort 5000:30090/TCP.

```
NAME                                          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
student-platform-deployment-7c6c7fdbb-j7kp8   1/1     Running   0          28s   10.244.0.4   minikube   <none>           <none>
student-platform-deployment-7c6c7fdbb-twq7n   1/1     Running   0          28s   10.244.0.5   minikube   <none>           <none>
student-platform-deployment-7c6c7fdbb-zqxdv   1/1     Running   0          28s   10.244.0.3   minikube   <none>           <none>
NAME              TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
student-service   NodePort   10.105.237.167   <none>        5000:30090/TCP   28s
(venv) shubham@LAPTOP-S8OAHEBU:~/student-academic-platform$
```

Figure 18: Three healthy replicas and the NodePort service

8.2 Self-Healing Proof

Pod student-platform-deployment-7c6c7fdbb-j7kp8 was deleted. Kubernetes immediately created a replacement pod (26sgq, age 20 s), and the replica count returned to three running pods without manual action.

```
NAME                                          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
student-platform-deployment-7c6c7fdbb-j7kp8   1/1     Running   0          12m   10.244.0.4   minikube   <none>           <none>
student-platform-deployment-7c6c7fdbb-twq7n   1/1     Running   0          12m   10.244.0.5   minikube   <none>           <none>
student-platform-deployment-7c6c7fdbb-zqxdv   1/1     Running   0          12m   10.244.0.3   minikube   <none>           <none>
NAME              TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
student-service   NodePort   10.105.237.167   <none>        5000:30090/TCP   12m
(venv) shubham@LAPTOP-S8OAHEBU:~/student-academic-platform$ kubectl delete pod student-platform-deployment-7c6c7fdbb-j7kp8
pod "student-platform-deployment-7c6c7fdbb-j7kp8" deleted from default namespace
(venv) shubham@LAPTOP-S8OAHEBU:~/student-academic-platform$ kubectl get pods
NAME                                          READY   STATUS    RESTARTS   AGE
student-platform-deployment-7c6c7fdbb-26sgq   1/1     Running   0          20s
student-platform-deployment-7c6c7fdbb-twq7n   1/1     Running   0          14m
student-platform-deployment-7c6c7fdbb-zqxdv   1/1     Running   0          14m
(venv) shubham@LAPTOP-S8OAHEBU:~/student-academic-platform$
```

Figure 19: Pod deletion and automatic replacement by Kubernetes

8.3 Accessing the Application

With the Docker driver on Linux, the service is reached through a tunnel opened by minikube service student-service --url. The command prints a local URL and must stay running in its own terminal. A second terminal then calls the health endpoint through that URL. The tunnel port is assigned each time the command starts, so the port number differs between runs.

```
shubham@LAPTOP-S8OAHEBU:~/student-academic-platform$ minikube service student-service --url
http://127.0.0.1:34503
❗  Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Figure 20: Service URL from minikube service student-service --url

```
shubham@LAPTOP-S8OAHEBU:~$ curl http://127.0.0.1:40853/api/health
{"service":"student-academic-platform","status":"UP"}
shubham@LAPTOP-S8OAHEBU:~$
```

Figure 21: Health check through the Kubernetes service: status UP

The application root page also loads in the browser and points to /api/health.



Figure 22: Application root page served from the Kubernetes service

9. Challenges and Solutions

Challenge	Cause	Solution
curl failed right after docker run	Gunicorn had not finished starting (container up for less than a second)	Waited a few seconds (sleep 5) and retried
Image not found by pods	imagePullPolicy: Never requires the image inside Minikube	Ran eval $(minikube docker-env) and rebuilt the image there
Service URL stopped working	The Minikube tunnel closes when the command is interrupted and the port changes	Kept minikube service --url running in one terminal and used the new port from another
Shell errors on pasted commands	Placeholders such as <url> and <pod-name> were typed literally	Replaced placeholders with real values

10. Architectural Reflections

- **Separation of concerns:** Business logic lives in services, HTTP handling in routes, and data in a separate database layer, which made each part easy to test on its own.
- **Quality gates in CI:** Linting, tests and the image build run on every pipeline run, so a failure at any stage stops the build before an image is produced.
- **Immutable artifacts:** A tagged, non-root container image runs the same way on a laptop, in Jenkins and in Kubernetes.
- **Health as a contract:** The /api/health endpoint serves Docker checks, Kubernetes probes and manual verification.
- **Declarative orchestration:** The Deployment keeps three replicas running, and the Service gives them one stable entry point. Failed pods are replaced automatically.
- **Known limitation:** The in-memory database resets on restart and is not shared between replicas, so a production version would need a persistent external database.

11. Learnings

- **Version control:** One branch per topic with pull requests gives a clean, reviewable history.
- **Continuous integration:** Automated lint and test stages in Jenkins give fast, repeatable feedback and produce reports as evidence.
- **Containerization:** Docker packages the application with its dependencies, and building inside Minikube avoids a registry for local work.

- **Orchestration and self-healing:** Kubernetes maintains the desired state, replaces failed pods and exposes the app through a Service.

12. Conclusion

The Student Record & Academic Management Platform was delivered end to end: source code on GitHub with six merged pull requests, 14 passing automated tests, a successful multi-stage Jenkins pipeline, a Docker image, and a three-replica Kubernetes deployment that recovers automatically from pod failure. All deliverables listed in the solution document are covered with evidence in this report.